

Tuan Anh Nguyen

Senior Unity / Mobile Game Developer · Performance & Live-Ops

Hanoi, Vietnam · Open to relocation · talama191@gmail.com · +84 838 331 032

linkedin.com/in/tuan-anh-nguyen-31648513a · github.com/talama191

SUMMARY

Senior Unity / C# developer with 6 years shipping and running live mobile titles at scale, including *Lumber Inc* (100M+ downloads on Google Play), where I'm second-in-command on the project, mentoring 3 developers. Strongest on performance: profiling-led CPU, memory and draw-call optimization across device tiers, plus rewriting systems a live game has outgrown. Examples: a 3D storage system held under ~10 KB warm-up across thousands of objects, ~43% load-time reduction on low-end Android. Delivers live-ops features end to end, architected as true A/B tests rather than just tuning numbers, and builds the tooling, automated testing and CI/CD pipeline behind them, AI-agent-assisted (Claude Code, Unity MCP) included. Open to relocation.

Shipped: *Lumber Inc* (AppQuantum), 100M+ downloads on Google Play · *Milk Farm Tycoon* (Eastside Games) · *Sport Stars* (collection RPG)

SKILLS

Languages: C#, TypeScript, Java, SQL, JavaScript

Performance: CPU & memory profiling, allocation/GC reduction, draw-call reduction (GPU instancing, texture batching, atlasing, canvas splitting), load-time optimization

Unity: UGUI / Canvas, Event System, Graphic Raycaster, DOTween, Animator, Addressables, UniTask, HLSL, Editor scripting, Unity Profiler, Unity Test Framework, Firebase (Remote Config, Analytics, Crashlytics), Google Play Games Services (Saved Games), Beamable (client SDK), Git

Build & CI: GitLab CI/CD and runners (Windows, macOS), headless Unity (batchmode) builds, Bash, xcodebuild, Android and iOS release signing, TestFlight upload

AI-Assisted Development: Claude Code, Unity MCP, used for scripting, internal tooling and test authoring; own architecture, code quality and review regardless of source

EXPERIENCE

Nopowerup, Senior Game Developer

Hanoi, Vietnam · Dec 2022 – Present

Mobile simulation and tycoon games. Unity, C#.

Lumber Inc, idle tycoon, publisher: AppQuantum · 100M+ downloads on Google Play · Aug 2024 – Present

play.google.com/store/apps/details?id=com.lumber.inc (not available on Vietnam store)

- Second-in-command on the project since the prior lead moved to another project, mentoring 3 developers and deciding most day-to-day technical questions, escalating the genuinely hard ones.
- Client-side feature delivery on a live build: the shop, limited-time events, IAP offers and the piggy-bank system, each with the UI it carries. Wired up Remote Config so live-ops can switch content on and retune rewards and economy without a new release.
- Built the forklift coordination system matching storage contents to incoming orders, architected with 3 coexisting order-fulfillment strategies as true A/B tests, not just tuning numbers, plus several interchangeable logistics-routing strategies, selected by Remote Config and swapped on restart, with losing variants cut once the data is in.
- Built the UI framework the game's screens sit on, including a redirect system: given a destination, it closes whatever popups are open, switches to the correct tab and scrolls the target into view, so a promotion lands on the exact thing being promoted rather than on a menu.
- Rewrote the live game's package-storage system to hold warm-up under ~10 KB allocation and a few ms per query across thousands of boxes and 40-50 storage areas. It had shipped as a 2D row/column layout; when the publisher needed non-uniform product shapes that layout couldn't represent, I replaced it with true 3D stacking plus a caching strategy.
- Built Google Play Games cloud save: every cloud save is mirrored to Play Games, so players who never linked an account get their progress back after a reinstall or on a new phone. When a device save and a Play Games save both exist, a chooser compares their progress and the player picks, at boot or right after signing in, so neither is ever overwritten silently.

- Profiling-led CPU and memory passes on the simulation loop. Draw-call reduction via GPU instancing and texture batching, plus canvas splitting, redraw batching and atlasing where the UI itself was the cost.
- Built the project's GitLab CI/CD pipeline, replacing a deprecated one: headless Unity jobs on Windows and Mac runners produce Android and iOS debug, signed release and low-end asset-variant builds, push Addressables bundles to the content server and upload iOS builds to TestFlight, after checking the version code against the last release tag. Each build runs as two Unity passes so define changes never force a mid-build recompile.
- Built a scenario-based automated test framework: testers compose scenarios from steps (set simulation stage, set currency, trigger an unlock, open a popup, assert a state) and can write and add their own scripts, extending coverage without an engineer in the loop, used by the test team. Unit tests on the systems that warrant them.
- Built a Claude Code-assisted tool that duplicates the limited-time-event structure (scripts, ScriptableObject config, UI bindings) into a new event shell, cutting setup time on new events. A personal workflow tool, not yet rolled out to the team.
- Built internal tools that give designers and testers self-serve control: a tap-triggered debug menu for testers to change game state on the device build, a Google-Sheets-to-Unity balance export pipeline for designers, a mesh combiner, and Editor QoL scripts. Separately, an Addressables duplication checker for the dev team, catching direct references to assets that should be Addressable-only before they silently duplicate into the base build.
- One of two regular code reviewers alongside the tech lead, mostly juniors and occasionally peer or senior work at the lead's request. Onboard new joiners with a codebase walkthrough and pair-programming on their first task.
- Triage live crashes from Firebase Crashlytics. First port of call from QA, designers and artists when something breaks, with a confirmed turnaround on the spot.
- Wrote small HLSL effects, for example lumber dissolving as it enters a machine. Traced an iOS-only rendering bug to a shader using half4, where the lower precision surfaced as visible pixel artifacts on device.

Milk Farm Tycoon, idle tycoon, publisher: Eastside Games · *Feb 2023 – Aug 2024*

play.google.com/store/apps/details?id=com.eastsidegames.milking

Runs on a Beamable backend: player state is held server-side rather than in a local save.

- Built the conveyor and distribution simulation.
- Built the battle pass system, the transporter-renting system (players rent trucks to deliver customer orders), and in-game localization, all against the Beamable client SDK, so progression, rental state and economy data persist server-side.
- CPU optimization on the simulation loop via targeted profiling passes, and draw-call reduction via GPU instancing and texture batching.
- Built a fake-shadow-mesh tool to help the team place shadow meshes faster.

Puzzle prototype · *Dec 2022 – Feb 2023*

- Prototyped a puzzle game in the style of *Where's My Perry?*, with sand in place of the liquid. Player swipes carve away sand so balls can fall through the cuts. Used triangulation to rebuild the sand mesh at runtime.

CodeGym, Unity Instructor (Part-time)

Hanoi, Vietnam · Aug 2023 – Aug 2024

Taught Unity / C# game programming part-time, concurrent with the Nopowerup role.

- Taught two cohorts of aspiring developers through Unity and C# game development, from programming fundamentals to building their own playable projects, reviewing their work and mentoring one-on-one to unblock progress.

Rainbow Studio, Game Developer

Hanoi, Vietnam · Aug 2020 – Aug 2022

Mobile hyper-casual games (Unity, C#) and web games (Egret engine, TypeScript).

- Gameplay and UI implementation across multiple small titles, plus internal tools for processing raw sprite, animation and game data.
- Ad-network and IAP SDK integration.

SIDE PROJECT

Sport Stars, Unity mobile collection RPG in a sports setting

Aug 2023 – Present

play.google.com/store/apps/details?id=com.bruhs.sportstars

A self-directed, co-developed, shipped side project. Players collect and level character cards carrying rarity-tagged, rolled abilities and per-discipline stats, build squads, and push a campaign plus championship and gauntlet ladders, funded by chests, missions and reward tracks. Three years in and roughly 1,800 commits, the second-largest contributor on the codebase. Data-driven, ScriptableObject-based architecture.

- Built the RPG layer: the character stats-and-abilities system, where rarity-tagged ability assets roll onto a card and resolve into per-match runtime behaviours, and the real-time match simulation those stats and abilities drive across eight disciplines.
- Built the HQ management sim where players train those characters: facility logistics with an autonomous agent AI that routes each card through a designer-authored routine, the amenity system, and the training assignment that grows each character's stats. Also built the mission and reward system, and a UI redirect that sends the player straight to a target screen.
- Reduced load time on low-end Android (Samsung A07-class hardware) from ~35 s to ~20 s, roughly 43% on the device class where load time hurts most. CPU optimization across the update loop.
- Built designer-facing balance tooling: Google Sheets with an export script that outputs code designers paste straight into Unity.

EDUCATION & LANGUAGES

Hanoi University, B.Sc. Computer Science (Software Engineering), *Aug 2018 – Jun 2023*

English: C1 (professional working proficiency) · **Vietnamese:** native